



Department Department of Electrical & Electronic Engineering

Rajshahi University of Engineering & Technology

LAB Report

Course Code : **EEE-3100**

Course Title : **Electronics Shop Practice**

Experiment No. : 08

Experiment Name : Design and Implementation of a Digital Voltmeter Using Arduino

Submitted By:

Name : Labib Marwan Hoque

Roll : 2101064

Section : B

Submitted To:

Asif Zaman Rizve

Lecturer

Department of Electrical & Electronic
Engineering

Rajshahi University of Engineering & Technology

Date of Experiment : 26 May,2025

Date of Submission : 16 June,2025

Experiment No.: 08

Experiment Name: Design and Implementation of a Digital Voltmeter Using Arduino.

Objective: -

- Understand the working principle of a digital voltmeter using Arduino.
- Interface a voltage sensor with Arduino for safe voltage measurement.
- Read analog voltage values using Arduino's ADC (Analog-to-Digital Converter).
- Display measured voltage on an LCD or Arduino Serial Monitor.
- Calibrate the voltmeter for accurate voltage readings.
- Analyze the limitations of Arduino-based voltage measurement (ADC resolution, range, accuracy).
- Apply the voltmeter in practical circuits.
- Learn the importance of voltage divider networks in measuring higher voltages.
- Develop coding skills for real-time data acquisition and display.
- Enhance troubleshooting skills in circuit connections and programming errors.

Theory:

A digital voltmeter is an instrument used to measure electrical potential difference between two points in a circuit. Traditional voltmeters are analog, but modern systems often use microcontrollers like Arduino to create digital voltmeters with higher precision and additional features.

The Arduino board has a built-in Analog-to-Digital Converter (ADC) that reads voltage values from analog input pins. The ADC converts the analog voltage into a digital value (ranging from 0 to 1023 for a 10-bit ADC). Since the Arduino's analog inputs can only handle voltages up to 5V (or 3.3V on some boards), a voltage divider circuit is used to measure higher voltages safely.

A voltage divider consists of two resistors connected in series. The output voltage (V_{out}) is taken from the junction between them and is calculated using the formula:

$$V_{out} = V_{in} \times (R2 / (R1 + R2))$$

Where V_{in} is the input voltage, and $R1$ and $R2$ are the resistances. By selecting appropriate resistor values, the input voltage can be scaled down to a range readable by the Arduino.

The Arduino reads the scaled-down voltage from the divider and converts the ADC value back to the actual voltage using the formula:

$$\text{Voltage} = (\text{ADC_Value} \times \text{Reference_Voltage}) / \text{ADC_Resolution}$$

For a 5V reference and 10-bit ADC (1024 steps), the formula becomes:

$$\text{Voltage} = (\text{ADC_Value} \times 5.0) / 1024$$

To measure higher voltages accurately, calibration may be required to account for resistor tolerances and ADC non-linearity. The measured voltage can be displayed on an LCD or the Serial Monitor for real-time monitoring.

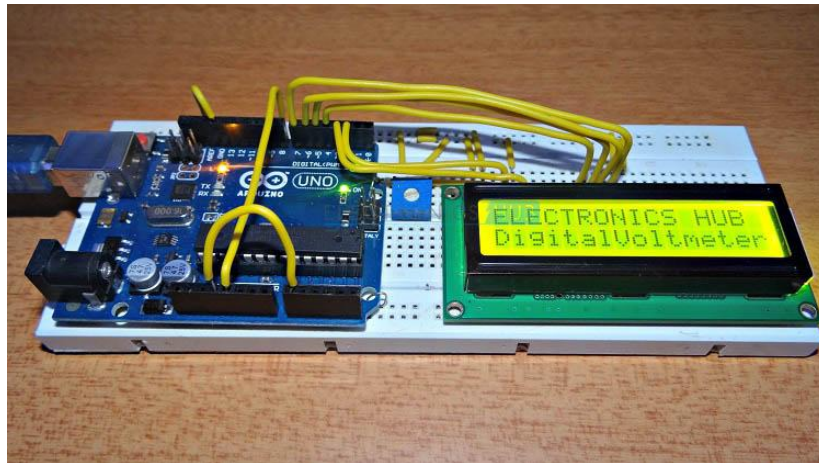


Fig.08.1: Digital Arduino Voltmeter

Required Apparatus:**Table 8.1: Table For Required Apparatus**

Sl. No.	Components	Specification	Quantity
1.	Arduino Uno	• ATmega328P • Operating Voltage: 5 V • Input Voltage: 7–12 V • Input Voltage: 6–20 V • Digital I/O Pins: 14 • PWM Digital I/O Pins: 6 (Pins 3, 5, 6, 9, 10, 11) • Analog Input Pins: 6 (A0 to A5) • DC Current per I/O Pin: 20 mA • DC Current for 3.3 V Pin: 50 mA • Flash Memory: 32 KB (ATmega328P) of which 0.5 KB used by bootloader • SRAM: 2 KB (ATmega328P) • EEPROM: 1 KB (ATmega328P) • Clock Speed: 16 MHz • USB Connection: Type B • LED Built-in: Pin 13	1
2.	Bluetooth Module	• HC-05 • Operating Voltage: 3.6-6V DC	1
3.	LCD Display	16×2 LCD	1
4.	I2C Module	I2C Address: 0×27	1
5.	Resistance	10K Ω and 100 K Ω	1
6.	DC Power	15V	-
7.	Jumper Wires	-	-
8.	IDE Software	• Arduino IDE	-

Circuit Diagram:

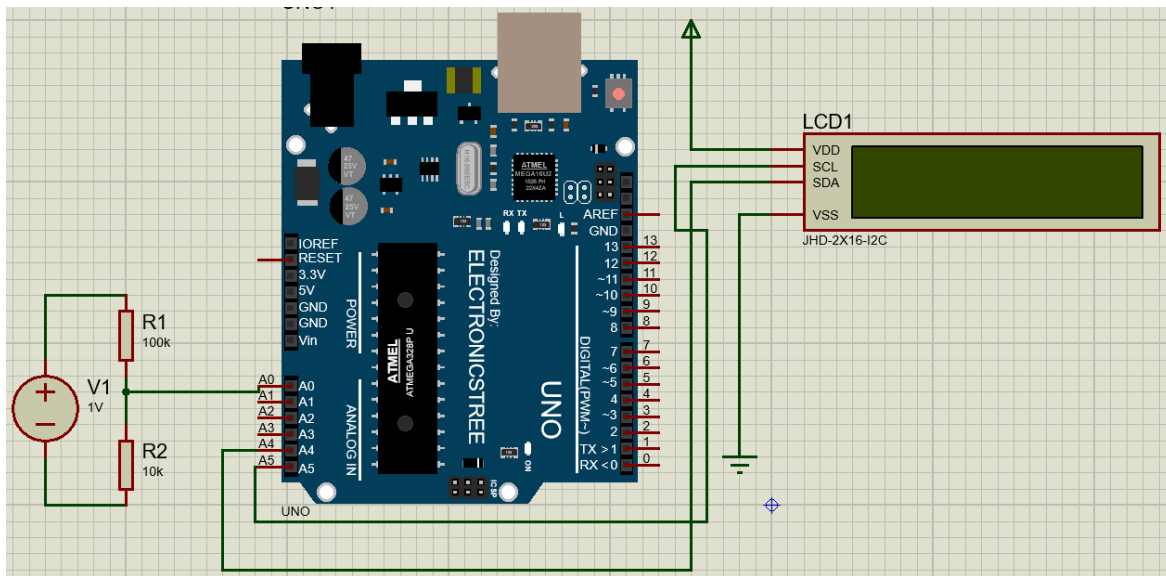


Fig.08.2: Circuit Diagram of Digital Arduino Voltmeter

Experimental Setup:

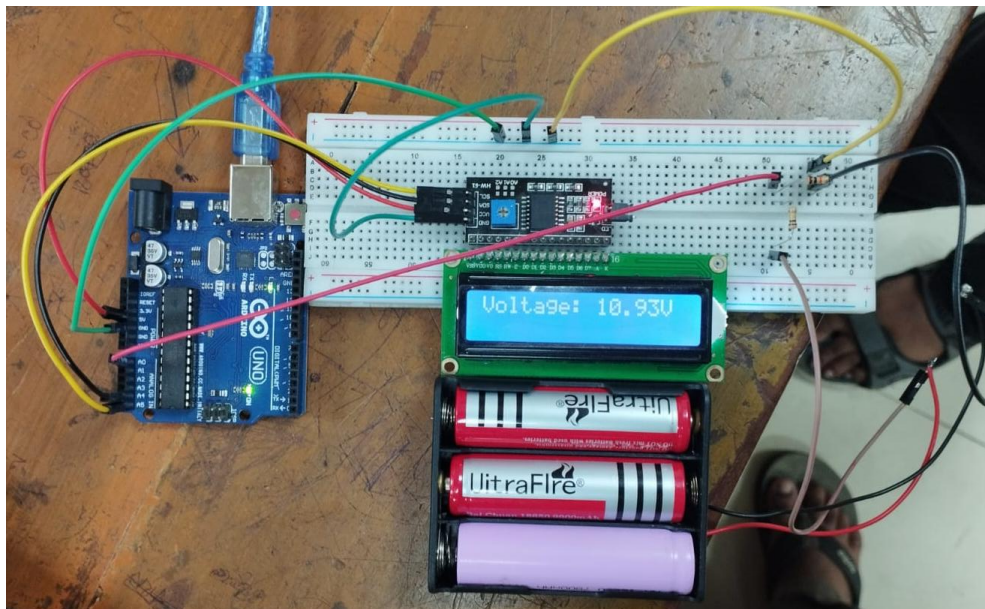


Fig.08.3: Experimental setup of the Digital Voltmeter

Code:

<pre>#include <Wire.h> #include <LiquidCrystal_I2C.h> // Set the LCD address and dimensions LiquidCrystal_I2C lcd(0x27, 16, 2); // 0x27 is the I2C address const float VREF = 5.0; // Reference voltage of Arduino const int ANALOG_MAX = 1023; // Max ADC value const float SCALE_FACTOR = 10.96; // Scale factor from voltage divider void setup() { lcd.begin(); // Initialize LCD (no arguments in your library version) lcd.backlight(); // Turn on the backlight lcd.setCursor(0, 0); lcd.print("Digital Voltmeter"); delay(2000); // Wait 2 seconds before measuring lcd.clear(); }</pre>	<pre>void loop() { int analog_value = analogRead(A0); // Read analog input (0-1023) float voltage = (analog_value * VREF) / ANALOG_MAX; // Convert to 0- 5V float actual_voltage = voltage * SCALE_FACTOR; // Scale to real voltage // Display voltage lcd.setCursor(0, 0); lcd.print("Voltage: "); // Clear previous value space lcd.setCursor(9, 0); lcd.print(actual_voltage, 2); // Print with 2 decimal places lcd.print("V"); delay(500); // Update every 0.5 seconds }</pre>
---	--

Experimental Procedure:

- The voltage divider circuit is constructed:
 - A 100k Ω resistor (R1) and a 10k Ω resistor (R2) are connected in series.
 - The junction between R1 and R2 is connected to Arduino analog pin A0.
 - The input voltage (Vin) is applied across R1, and R2 is grounded.
- The LCD module is connected to the Arduino:
 - For parallel LCD (16 $\times$ 2):
 - RS pin is connected to Arduino pin 2.
 - EN pin is connected to Arduino pin 3.
 - D4-D7 pins are connected to Arduino pins 4, 5, 6, 7 respectively.
 - VSS and RW pins are grounded.
 - VDD is connected to 5V.
 - For I2C LCD:
 - SDA is connected to Arduino A4 (or SDA pin).
 - SCL is connected to Arduino A5 (or SCL pin).
 - VCC is connected to 5V, and GND is grounded.
- The Arduino is powered:
 - The Arduino Uno is connected to a PC via USB.
 - The voltage divider input is connected to a test voltage source (e.g., a 9V battery).

- The Arduino code is uploaded:
 - The provided voltmeter sketch is opened in the Arduino IDE.
 - The correct board (Arduino Uno) and port are selected.
 - The code is compiled and uploaded to the Arduino.
- The voltmeter is tested:
 - The LCD display is observed for voltage readings.
 - A multimeter is used to measure the actual input voltage for comparison.
 - If readings are inaccurate, the voltage divider ratio in the code is adjusted.
- Measurements are recorded:
 - Different DC voltage levels (e.g., 5V, 9V, 12V) are applied.
 - The Arduino's readings and multimeter readings are recorded in a table.
- The setup is dismantled:
 - All connections are safely disconnected.
 - Components are stored properly.

Task No.: 01

Task Name: Wireless Voltage Monitoring System via Bluetooth Communication

Objectives:

- To design and implement a real-time voltage monitoring system that transmits measured DC voltage from an Arduino to a smartphone using a Bluetooth module (HC-05).
- To display the voltage data on a mobile application via serial communication, enabling wireless monitoring without a physical display unit.
- To explore the integration of sensor data acquisition with wireless transmission for enhanced portability and usability in electrical measurements.

Code:

<pre>const int analogPin = A0; float voltage = 0.0; int rawValue = 0; void setup() { Serial.begin(9600); } void loop() { rawValue = analogRead(analogPin);</pre>	<pre>voltage = (rawValue * 5.0) / 1023.0; Serial.print("Voltage: "); Serial.print(voltage); Serial.println(" V"); delay(1000); }</pre>
---	--

Task No.: 02

Task Name: EEPROM-Integrated Voltage Logger with Real-Time LCD and Serial Display

Objective:

- To build a microcontroller-based voltage monitoring system that reads analog voltage values and logs them to internal EEPROM in regular intervals.
- To display the current voltage in real time on an LCD screen.
- To output each stored voltage reading along with its EEPROM address to the Serial Monitor for debugging and offline data verification.

Code-1: Logging Voltage to EEPROM .

<pre>#include <Wire.h> #include <LiquidCrystal_I2C.h> #include <EEPROM.h> LiquidCrystal_I2C lcd(0x27, 16, 2); const int analogPin = A0; const float VOLTAGE_DIVIDER_RATIO = 11.0; int address = 0; float readAverageVoltage() { long sum = 0; const int samples = 10; for (int i = 0; i < samples; i++) { sum += analogRead(analogPin); delay(10); } return (sum / samples) * 5.0 / 1023.0; } void setup() { analogReference(DEFAULT); lcd.begin(); lcd.backlight(); lcd.setCursor(0, 0); lcd.print("Logging Started"); delay(2000); lcd.clear(); Serial.begin(9600); Serial.println("Logging to EEPROM..."); }</pre>	<pre>void loop() { float voltage = readAverageVoltage(); float inputVoltage = voltage * VOLTAGE_DIVIDER_RATIO; // Display on LCD lcd.setCursor(0, 0); lcd.print("Voltage: "); lcd.setCursor(0, 1); lcd.print(inputVoltage, 2); lcd.print(" V "); // Store in EEPROM (float = 4 bytes) if (address + 4 <= EEPROM.length()) { EEPROM.put(address, inputVoltage); Serial.print("Stored "); Serial.print(inputVoltage, 2); Serial.print(" V at addr "); Serial.println(address); address += 4; } else { Serial.println("EEPROM full"); lcd.setCursor(0, 0); lcd.print(" EEPROM FULL"); } delay(2000); // Log every 2 seconds }</pre>
--	---

Code-2: Reading Voltage from EEPROM via Serial Monitor

<pre>#include <EEPROM.h> float readVoltageFromEEPROM(int addr) { float value = 0; if (addr >= 0 && addr + 4 <= EEPROM.length()) { EEPROM.get(addr, value); } return value; } void setup() { Serial.begin(9600); Serial.println("EEPROM Reader Ready"); Serial.println("Enter address (multiple of 4):"); } void loop() { if (Serial.available()) { String input = Serial.readStringUntil('\n'); input.trim(); int addr = input.toInt();</pre>	<pre> if (addr % 4 != 0 addr < 0 addr + 4 > EEPROM.length()) { Serial.println("Invalid address. Must be a multiple of 4 and within range."); } else { float storedVoltage = readVoltageFromEEPROM(addr); Serial.print("Voltage at address "); Serial.print(addr); Serial.print(" = "); Serial.print(storedVoltage, 2); Serial.println(" V"); } Serial.println("\nEnter another address:"); } }</pre>
---	--

Results:

Task 01: Wireless Voltage Monitoring System via Bluetooth Communication

In this task, a real-time digital voltmeter was implemented using Arduino and an HC-05 Bluetooth module. The system successfully measured analog voltage input and transmitted the corresponding voltage values wirelessly to a smartphone via Bluetooth. The readings were accurately displayed on a mobile app, demonstrating effective remote voltage monitoring without the need for a physical display.

Task 02: EEPROM-Integrated Voltage Logger with Real-Time LCD and Serial Display

This task involved designing a standalone voltage monitoring system capable of logging voltage data into the Arduino's internal EEPROM. The current voltage was displayed on an I2C LCD in real time, while logged values along with EEPROM addresses were sent to the Serial Monitor. The system effectively stored and retrieved voltage data, supporting persistent data logging for later analysis.

Justification of CO2 and PO(j):

To safely measure higher voltages with the Arduino, we used a voltage divider to step them down into the ADC's readable range. We then calibrated the output to correct for any hardware inconsistencies. With an LCD for display, Bluetooth for wireless monitoring, and

EEPROM for saving past readings, our voltmeter offers real-time data you can trust—while keeping a history even when it's not connected.
--

Conclusion:

This project showed how simple components and smart coding can build a reliable digital voltmeter with real-time and wireless features. It deepened our understanding of embedded systems and proved how adaptable Arduino can be for practical electrical measurements.

Reference:

<https://www.electronicshub.org/digital-arduino-voltmeter/>